

RAPPORT DE PROJET

Conception et développement d'une API REST

Gestion des étudiants — Flask / MySQL

Base de données : ciel2027

1. Description du projet

Ce projet consiste à développer une API REST permettant la gestion des étudiants d'un établissement : consultation, ajout, modification et suppression des fiches étudiants. L'API est développée avec le micro-framework Python Flask et s'appuie sur une base de données MySQL (base « ciel2027 », table « etudiant ») pour la persistance des données.

La ressource principale manipulée est l'étudiant, caractérisé par : un identifiant (idetudiant), un nom, un prénom, un email et un numéro de téléphone. L'ensemble des opérations est exposé sous forme de endpoints HTTP suivant les principes REST (une ressource, un verbe HTTP par action).

1.1 Diagramme de cas d'utilisation

Le diagramme ci-dessous présente les interactions entre l'acteur principal (l'utilisateur de l'API, par exemple une application cliente) et les fonctionnalités offertes par le système. La base de données MySQL intervient comme acteur secondaire, sollicitée par chaque cas d'utilisation pour la lecture ou l'écriture des données.

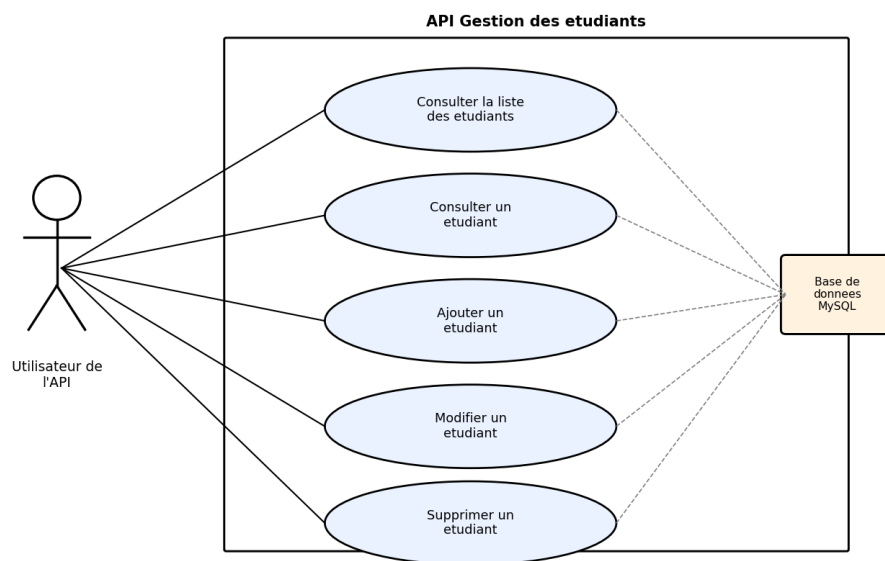


Figure 1 — Diagramme de cas d'utilisation de l'API Gestion des étudiants

2. Description de l'API

La documentation des endpoints ci-dessous a été rédigée dans l'esprit de la spécification OpenAPI/Swagger (swagger.io), qui est le standard de référence pour décrire une API REST : pour chaque route, la méthode HTTP, le corps de requête attendu et les réponses possibles sont précisés. Ce même contenu peut être formalisé dans un fichier openapi.yaml et visualisé avec Swagger UI.

2.1 Endpoints — Version 1 (/v1/etudiants)

Méthode	Route	Description	Corps de la requête (JSON)	Réponses
GET	/v1/etudiants/	Liste tous les étudiants	—	201 Created [{ idetudiant, nom, prenom, email, telephone }, ...] (anomalie : un GET réussi devrait renvoyer 200, pas 201)

Méthode	Route	Description	Corps de la requête (JSON)	Réponses
GET	/v1/etudiants/{id}	Récupère un étudiant par id	—	200 OK { idetudiant, nom, prenom, email, telephone } Si id inexistant : erreur 500 non gérée (TypeError non interceptée)
POST	/v1/etudiants/	Ajoute un étudiant	{ "nom", "prenom", "email", "telephone" }	201 Created { "message": "Ajout ok" } Aucune validation des champs
PUT	/v1/etudiants/{id}	Modifie un étudiant	{ "nom", "prenom", "email", "telephone" }	200 OK { "message": "Modification ok" }
DELETE	/v1/etudiants/{id}	Supprime un étudiant	—	200 OK { "message": "Suppression ok" }

2.2 Endpoints — Version 2 (/v2/etudiants)

Méthode	Route	Description	Corps de la requête (JSON)	Réponses
GET	/v2/etudiants/	Liste tous les étudiants	—	201 Created [{ idetudiant, nom, prenom, email, telephone }, ...]
GET	/v2/etudiants/{id}	Récupère un étudiant par id	—	200 OK — étudiant trouvé 404 Not Found { "erreur": "id invalide" } — id inexistant (géré par try/except)
POST	/v2/etudiants/	Ajoute un étudiant	{ "nom", "prenom", "email", "telephone" }	201 Created — { "message": "Ajout ok" } 404 Not Found — { "erreur": "erreur il manque une information" } (anomalie : un champ manquant devrait renvoyer 400 Bad Request)
PUT	/v2/etudiants/{id}	Modifie un étudiant	{ "nom", "prenom", "email", "telephone" }	200 OK { "message": "Modification ok" }
DELETE	/v2/etudiants/{id}	Supprime un étudiant	—	200 OK { "message": "Suppression ok" }

Toutes les réponses sont au format JSON, conformément à la convention REST. Le format d'échange (Content-Type: application/json) ainsi que les codes de statut HTTP renvoyés par chaque route sont ceux observés directement dans le code source et confirmés par les tests Postman (section 5).

3. Récapitulatif des versions de l'API

Trois étapes de développement ont été identifiées à partir des fichiers du projet, de la version préliminaire (squelette Flask) jusqu'à la version 2 actuelle :

Version	Contenu / évolutions	Limites identifiées
Version préliminaire	Squelette Flask (routes "/" et "/etudiants/" statiques) et script de test de connexion MySQL séparé. Aucune logique métier, aucun lien entre Flask et la base.	Pas encore d'API REST fonctionnelle.
Version 1 (v1)	Première API REST complète sous /v1/etudiants/ : GET (liste et par id), POST, PUT, DELETE, connectée directement à MySQL.	Requêtes SQL construites par concaténation de chaînes (injection SQL possible). Aucune gestion d'erreur : un id inexistant provoque une erreur serveur (500).
Version 2 (v2)	Reprend le CRUD de la v1 sous /v2/etudiants/ et ajoute une gestion des erreurs (try/except) sur la lecture par id et l'ajout, avec des messages JSON explicites.	Les requêtes SQL restent construites par f-string (injection SQL toujours possible). Codes HTTP parfois inadaptés (404 au lieu de 400). Mode debug=True actif.

4. Diagramme de classes — classe Database

Le code fourni gère la connexion à MySQL de façon procédurale (variables globales mydb et cursor). Pour produire un diagramme de classes exploitable avec pyreverse, cette logique a été encapsulée dans une classe Database regroupant la connexion, le curseur et les opérations CRUD (get_all_etudiants, get_etudiant, add_etudiant, update_etudiant, delete_etudiant).

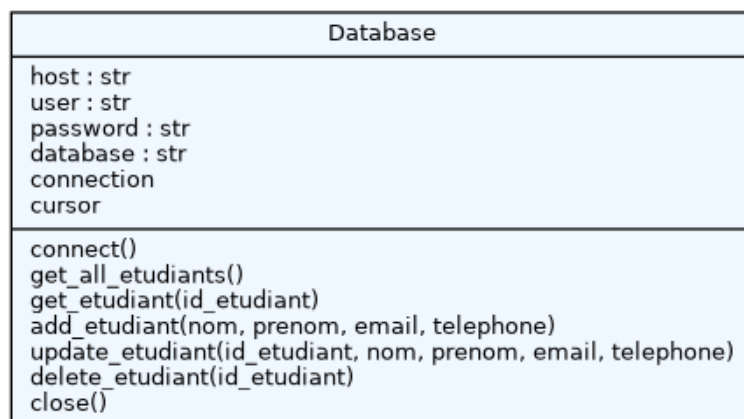


Figure 2 — Diagramme de classe UML de Database, généré à partir de database.py avec pyreverse (pylint.pyreverse)

Commande utilisée pour la génération : `pyreverse -o png -c Database database.py`

5. Fiche des tests de l'API (Postman)

Les tests suivants ont été réalisés avec Postman par un autre étudiant du binôme/groupe, afin de vérifier le bon fonctionnement des routes d'ajout, de modification et de suppression.

#	Requête testée	Corps envoyé	Résultat obtenu
1	POST http://127.0.0.1:5000/v2/etudiants	{ "email": "", "nom": "", "prenom": "toto", "telephone": "" }	201 Created — { "message": "Ajout ok" } en 6 ms (l'ajout est accepté même avec des champs vides — absence de validation)
2	PUT http://127.0.0.1:5000/v1/etudiants/14	{ "email": "", "nom": "titi", "prenom": "toto", "telephone": "+596696584215" }	200 OK — { "message": "Modification ok" } en 15 ms

#	Requête testée	Corps envoyé	Résultat obtenu
3	DELETE http://127.0.0.1:5000/v1/etudiants/14	—	200 OK — { "message": "Suppression ok" } en 9 ms

5.1 Captures d'écran des tests

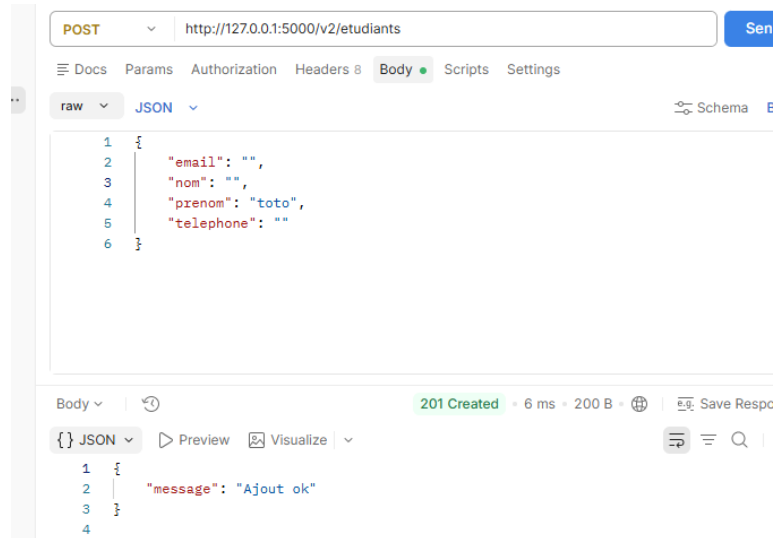


Figure 3 — Test POST /v2/etudiants : ajout d'un étudiant (201 Created)

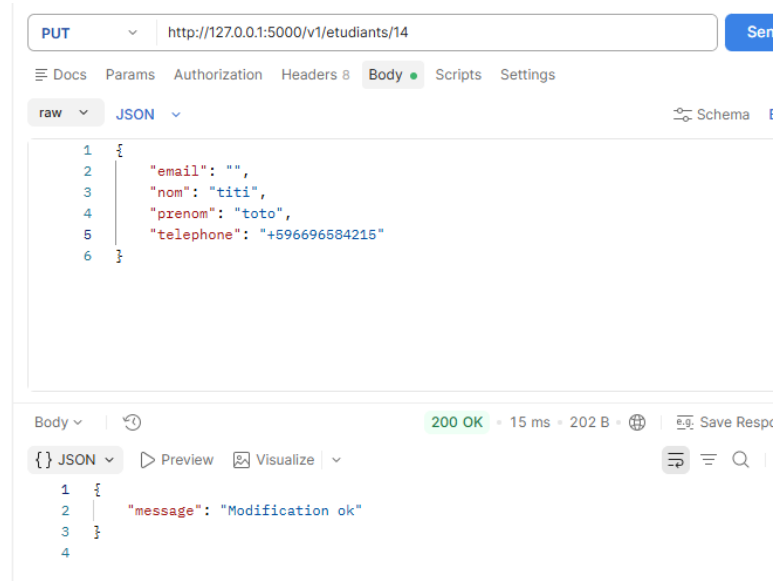


Figure 4 — Test PUT /v1/etudiants/14 : modification d'un étudiant (200 OK)

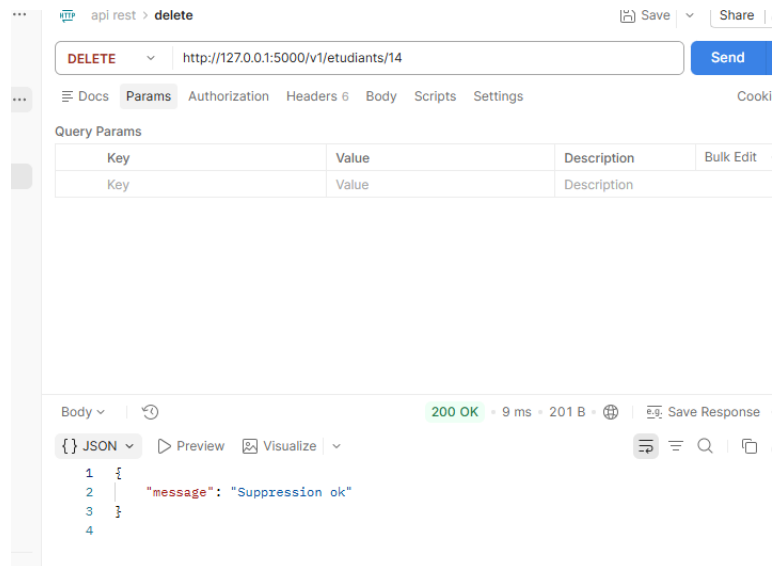


Figure 5 — Test DELETE /v1/etudiants/14 : suppression d'un étudiant (200 OK)

5.2 Vérification en base de données

Les résultats des tests ont été confirmés directement dans phpMyAdmin, où l'on retrouve les étudiants ajoutés lors des tests (lignes « titi/toto » et « toto ») aux côtés des étudiants initiaux.

✓ Affichage des lignes 0 - 8 (total de 9, traitement en 0,0004 seconde(s))

SELECT * FROM `etudiant`

☐ Profilage [Éditer en ligne](#) [\[Éditer \]](#) [\[Expliquer SQL \]](#) [\[Créer le code source PHP \]](#) [\[Actualiser \]](#)

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Options supplémentaires

	idetudiant	nom	prenom	email	telephone
☐ Éditer ☐ Copier ☐ Supprimer	1	ALEXANDRE	Thomas	thomas.alexandre97@gmail.com	+596696123456
☐ Éditer ☐ Copier ☐ Supprimer	2	BOUTRIN	Mélissa	melissa.boutrin@gmail.com	+596696234567
☐ Éditer ☐ Copier ☐ Supprimer	3	CHALONCYS	Stéphane	stephane.chaloncys@gmail.com	+596696345678
☐ Éditer ☐ Copier ☐ Supprimer	4	DUMONT	Aurèle	aurelie.dumont972@gmail.com	+596696456789
☐ Éditer ☐ Copier ☐ Supprimer	5	EUSTACHE	Yann	yann.eustache97@gmail.com	+596696501852
☐ Éditer ☐ Copier ☐ Supprimer	13	JEANDU	Emily	emilyjeandu@gmail.com	+596696805630
☐ Éditer ☐ Copier ☐ Supprimer	15	titi	toto		
☐ Éditer ☐ Copier ☐ Supprimer	16	toto			
☐ Éditer ☐ Copier ☐ Supprimer	17	toto			

☐ Tout cocher | Avec la sélection : [Éditer](#) [Copier](#) [Supprimer](#) [Exporter](#)

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Opérations sur les résultats de la requête

Figure 6 — Contenu de la table etudiant après exécution des tests

6. Conclusion et améliorations possibles

Ce projet a permis de mettre en place une API REST fonctionnelle pour la gestion des étudiants, avec une évolution progressive entre les versions 1 et 2 : l'ajout de blocs try/except en v2 constitue une première amélioration de la robustesse. Les tests Postman confirment que les opérations de création, modification et suppression fonctionnent comme attendu.

L'analyse du code met cependant en évidence plusieurs axes d'amélioration importants du point de vue de la cybersécurité :

- **Injection SQL** : toutes les requêtes sont construites par concaténation de chaînes (f-strings) avec les données reçues du client. Il est indispensable de passer par des requêtes paramétrées (curseur.execute(req, (valeurs,))) ou un ORM (SQLAlchemy) pour empêcher toute injection SQL.

- Authentification et autorisation : aucune authentification n'est actuellement exigée pour accéder aux routes. L'ajout d'une authentification (clé API ou JWT) permettrait de restreindre l'accès aux seuls utilisateurs autorisés.
- Chiffrement des échanges : l'API est exposée en HTTP (<http://127.0.0.1:5000>). En production, un déploiement en HTTPS (TLS) est nécessaire pour protéger les données transmises, notamment les informations personnelles des étudiants (RGPD).
- Validation des entrées : les tests montrent que des champs vides sont acceptés (POST avec nom et email vides). Une validation stricte des champs (format email, longueur, champs obligatoires) doit être ajoutée avant toute insertion en base.
- Gestion des secrets : les identifiants de connexion à la base (host, user, password) sont écrits en clair dans le code source. Ils devraient être externalisés via des variables d'environnement ou un gestionnaire de secrets.
- Mode debug : `app.run(debug=True)` est actif dans toutes les versions. Ce mode expose le débogueur interactif Werkzeug, qui peut permettre l'exécution de code arbitraire s'il reste activé en production ; il doit être désactivé (`debug=False`) hors développement.
- Codes de statut HTTP : certains codes ne respectent pas la sémantique REST (201 sur un GET, 404 pour une donnée manquante au lieu de 400). Une réponse d'erreur cohérente facilite l'intégration côté client et limite les fuites d'information sur l'implémentation interne.
- Journalisation et limitation de débit (rate limiting) : la mise en place de logs d'accès et d'un mécanisme anti-abus (ex. Flask-Limiter) permettrait de détecter et limiter les tentatives d'exploitation automatisées.

La correction de ces points, en priorité l'usage de requêtes paramétrées et l'ajout d'une authentification, constitue la suite logique du projet pour amener l'API d'un prototype fonctionnel vers un service exploitable en production.